

Basic LMM example

James Foster

2026-06-02

Contents

Basic Linear Mixed Model example	1
Simulate some data	1
Save and reload data	2
Plot the data	3
Fit the maximal model	4
Compare models	4
Inspect the maximal model ANOVA	5
Model reduction	5
Post-hoc comparisons	6
Prediction plots	7

```
library(lme4)
set.seed(17070523)
knitr::opts_chunk$set(echo = TRUE)
```

Basic Linear Mixed Model example

This example simulates a repeated-measures experiment and fits mixed-effects models using `lmer()`.

Simulate some data

We simulate responses for multiple animals, stimulus levels, and stimulus types.

```
n_animals <- 10
n_levels <- 7
n_types <- 2
replicates <- 20

pop_mean <- 5.00
pop_sd <- 1.00
slope_mean <- 3.00
slope_sd <- 1.03
```

```

type_mean <- 2.50
type_slope <- 3.5
type_animal_sd <- 0.55
extra_error_sd <- 0.10

animal <- rep(x = 1:n_animals, times = n_levels)
animal <- sort(animal)
animal <- rep(x = animal, times = n_types)
stimulus <- rep(x = 1:n_levels - 1, times = n_animals)
stimulus <- rep(x = stimulus, times = n_types)
stype <- rep(x = 1:n_types, times = n_animals * n_levels)
stype <- sort(stype)

animal <- rep(x = animal, times = replicates)
stimulus <- rep(x = stimulus, times = replicates)
stype <- rep(x = stype, times = replicates)

base_bias_animal <- rnorm(n = n_animals, mean = pop_mean, sd = pop_sd)
slope_animal <- rnorm(n = n_animals, mean = slope_mean, sd = slope_sd)
type_animal <- rnorm(n = n_animals, mean = 1.0, sd = type_animal_sd)

response_y <- rep(x = NA, length = n_animals * n_levels * n_types * replicates)
for (ii in seq_along(response_y)) {
  response_y[ii] <- base_bias_animal[animal[ii]] +
    (stype[ii] - 1) * type_mean +
    (slope_animal[animal[ii]] +
      (stype[ii] - 1) * type_slope * type_animal[animal[ii]]) * stimulus[ii] +
    rnorm(n = 1, mean = 0, sd = extra_error_sd)
}

sim_data <- data.frame(
  response_y = response_y,
  animal = LETTERS[animal],
  stimulus = stimulus,
  type = factor(stype, labels = c('alpha', 'beta'))
)

summary(sim_data)

```

```

##      response_y      animal      stimulus      type
## Min.   : 3.087   Length:2800   Min.    :0    alpha:1400
## 1st Qu.:10.514   Class  :character   1st Qu.:1    beta :1400
## Median :16.985   Mode   :character   Median :3
## Mean    :20.537                Mean    :3
## 3rd Qu.:27.067                3rd Qu.:5
## Max.    :65.279                Max.    :6

```

Save and reload data

We write the data to CSV and reload it for analysis, then format categorical variables.

```

root_dir <- file.path(dirname(knitr::current_input(dir = TRUE)), 'Simulated Data')
file_name <- 'simulated_data.csv'
write.csv(file = file.path(root_dir, file_name), x = sim_data, row.names = FALSE)

dta <- read.csv(file = file.path(root_dir, file_name), header = TRUE)
dta <- within(dta, {
  animal <- factor(animal)
  type <- factor(type)
})

summary(dta)

```

```

##      response_y      animal      stimulus      type
## Min.       : 3.087    A       : 280    Min.       :0    alpha:1400
## 1st Qu.:10.514    B       : 280    1st Qu.:1    beta :1400
## Median :16.985    C       : 280    Median :3
## Mean    :20.537    D       : 280    Mean    :3
## 3rd Qu.:27.067    E       : 280    3rd Qu.:5
## Max.     :65.279    F       : 280    Max.     :6
##                      (Other):1120

```

Plot the data

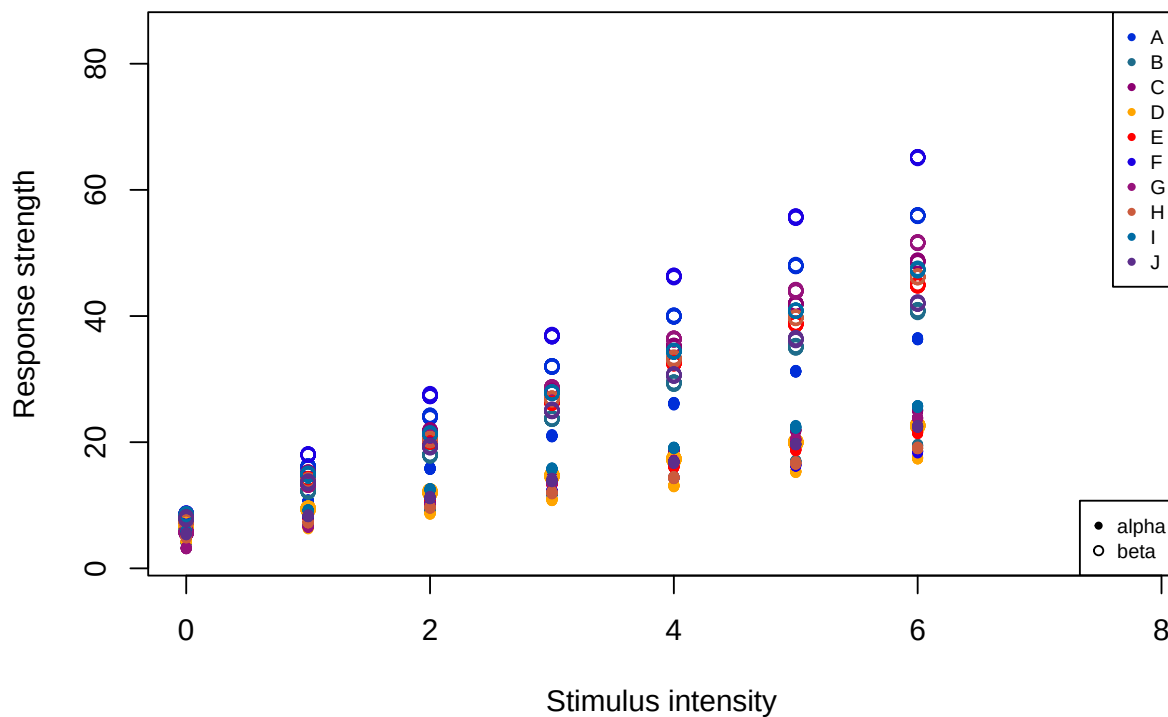
We visualise the individual animal responses by stimulus and type.

```

clz <- sapply(length(unique(dta$animal)), colorRampPalette(c('red', 'blue', 'cyan4', 'magenta4', 'orange4')),
              FUN = function(x) sample(1:x, x))
clz <- sample(clz)

plot(NULL,
      xlab = 'Stimulus intensity',
      ylab = 'Response strength',
      xlim = range(stimulus) * c(0.7, 1.3),
      ylim = range(response_y) * c(0.7, 1.3))
for (aa in unique(dta$animal)) {
  with(subset(dta, animal == aa), {
    points(
      x = stimulus,
      y = response_y,
      col = clz[which(LETTERS %in% aa)],
      pch = c(20, 21)[1 + type %in% levels(type)[2]]
    )
  })
}
legend('topright', legend = unique(dta$animal), col = clz, pch = 20, cex = 0.7)
legend('bottomright', legend = unique(dta$type), pch = c(20, 21), cex = 0.7)

```



Fit the maximal model

We fit a maximal linear mixed model with random intercepts and slopes for each animal.

```
ctrl_opt <- lmerControl(optimizer = 'bobyqa')
mixmod.max <- lmer(
  formula = response_y ~ stimulus * type + (1 + stimulus * type | animal),
  data = dta,
  control = ctrl_opt
)
```

```
## boundary (singular) fit: see help('isSingular')
```

```
mixmod.null <- lmer(formula = response_y ~ 1 + (1 | animal), data = dta)
```

Compare models

We compare the maximal model to the null model using AIC and a likelihood ratio test.

```
extractAIC(mixmod.max)[2]
```

```
## [1] -4706.033
```

```
extractAIC(mixmod.null)[2]
```

```
## [1] 22160.38
```

```
anova(mixmod.max, mixmod.null, test = 'Chisq')
```

```
## refitting model(s) with ML (instead of REML)
```

```
## Data: dta
```

```
## Models:
```

```
## mixmod.null: response_y ~ 1 + (1 | animal)
```

```
## mixmod.max: response_y ~ stimulus * type + (1 + stimulus * type | animal)
```

```
##          npar    AIC    BIC logLik -2*log(L) Chisq Df Pr(>Chisq)
```

```
## mixmod.null      3 22160 22178 -11077      22154
```

```
## mixmod.max      15 -4706 -4617   2368      -4736 26890 12 < 2.2e-16 ***
```

```
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Inspect the maximal model ANOVA

```
anova(mixmod.max)
```

```
## Analysis of Variance Table
```

```
##          npar  Sum Sq Mean Sq    F value
```

```
## stimulus      1   83.29   83.29   8702.828
```

```
## type          1 1225.11 1225.11 128014.147
```

```
## stimulus:type  1    0.40    0.40    42.312
```

Model reduction

We test whether interaction, stimulus, or type can be removed without worsening model fit.

```
no_int <- update(mixmod.max, . ~ . - stimulus:type -
```

```
                (1 + stimulus * type | animal) +
```

```
                (1 + stimulus + type | animal))
```

```
anova(mixmod.max, no_int)
```

```
## refitting model(s) with ML (instead of REML)
```

```
## Data: dta
```

```
## Models:
```

```
## no_int: response_y ~ stimulus + type + (1 + stimulus + type | animal)
```

```
## mixmod.max: response_y ~ stimulus * type + (1 + stimulus * type | animal)
```

```
##          npar    AIC    BIC logLik -2*log(L) Chisq Df Pr(>Chisq)
```

```
## no_int      10 15687 15746 -7833.4      15667
```

```
## mixmod.max   15 -4706 -4617   2368.0      -4736 20403  5 < 2.2e-16 ***
```

```
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```

no_stimulus <- update(no_int, . ~ . - stimulus -
                      (1 + stimulus + type | animal) +
                      (1 + type | animal))
anova(no_int, no_stimulus)

## refitting model(s) with ML (instead of REML)

## Data: dta
## Models:
## no_stimulus: response_y ~ type + (1 + type | animal)
## no_int: response_y ~ stimulus + type + (1 + stimulus + type | animal)
##          npar    AIC    BIC  logLik -2*log(L)  Chisq Df Pr(>Chisq)
## no_stimulus      6 21175 21211 -10581.7      21163
## no_int           10 15687 15746 -7833.4      15667 5496.6  4 < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

no_type <- update(no_stimulus, . ~ . - type -
                  (1 + type | animal) + (1 | animal))
anova(no_stimulus, no_type)

```

```

## refitting model(s) with ML (instead of REML)

## Data: dta
## Models:
## no_type: response_y ~ (1 | animal)
## no_stimulus: response_y ~ type + (1 + type | animal)
##          npar    AIC    BIC logLik -2*log(L)  Chisq Df Pr(>Chisq)
## no_type      3 22160 22178 -11077      22154
## no_stimulus  6 21175 21211 -10582      21163 991.04  3 < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

Post-hoc comparisons

We use `emmeans` and `pbkrtest` to make pairwise comparisons for type and slope effects.

```

library(emmeans)
library(pbkrtest)

emm_intercepts <- emmeans(mixmod.max, specs = list(pairwise ~ type),
                          adjust = 'sidak')
summary(emm_intercepts)

## $'emmeans of type'
##   type emmean    SE df lower.CL upper.CL
## alpha  14.0 0.919  9     11.9     16.1
## beta   27.1 1.810  9     23.0     31.1
##
## Degrees-of-freedom method: kenward-roger

```

```
## Confidence level used: 0.95
##
## $'pairwise differences of type'
##      1      estimate    SE df t.ratio p.value
## alpha - beta      -13 1.62  9  -8.043  <.0001
##
## Degrees-of-freedom method: kenward-roger

emm_slopes_interact <- emtrends(mixmod.max, specs = list(pairwise ~ type),
                               var = 'stimulus', adjust = 'sidak')
summary(emm_slopes_interact)

## $'emmeans of type'
## type stimulus.trend    SE df lower.CL upper.CL
## alpha           2.98 0.280  9     2.35     3.61
## beta            6.49 0.562  9     5.22     7.76
##
## Degrees-of-freedom method: kenward-roger
## Confidence level used: 0.95
##
## $'pairwise differences of type'
##      1      estimate    SE df t.ratio p.value
## alpha - beta    -3.51 0.54  9  -6.505  0.0001
##
## Degrees-of-freedom method: kenward-roger
```

Prediction plots

We generate fitted predictions and plot the model with confidence intervals.

```
newdta <- with(dta, expand.grid(stimulus = unique(stimulus),
                              type = unique(type),
                              animal = unique(animal)))
prd <- predict(mixmod.max, newdata = newdta)

pfun <- function(x) predict(x, newdata = newdta)
avail.cores <- parallel::detectCores() - 1
clt <- parallel::makeCluster(spec = avail.cores, type = 'PSOCK')
parallel::clusterExport(clt, list('mixmod.max', 'dta', 'newdta'))

bt <- bootMer(
  mixmod.max,
  FUN = pfun,
  nsim = 100,
  re.form = NA,
  parallel = 'snow',
  ncpus = parallel::detectCores() - 1,
  cl = clt
)
parallel::stopCluster(clt)

param_q <- aggregate(pred ~ stimulus * type,
```

```

        data = with(bt, data.frame(newdta, pred = c(t(t)))),
        FUN = quantile,
        probs = c(0.025, 0.975))
mod_mean <- aggregate(prd ~ stimulus * type,
                      data = cbind(newdta, prd),
                      FUN = mean)
param_data <- merge(merge(newdta, param_q, all = TRUE), mod_mean)
mod_pred <- within(param_data, {
  mod_mean <- prd
  CI_02.5 <- pred[, '2.5%']
  CI_97.5 <- pred[, '97.5%']
  rm(list = c('pred', 'prd'))
})

with(dta, plot(x = stimulus, y = response_y,
              bg = adjustcolor(col = c('orange2', 'darkblue')[
                1 + type %in% 'alpha'], alpha.f = 0.5),
              col = 'black', pch = 21))
with(subset(mod_pred, type %in% 'alpha'), {
  polygon(x = c(sort(stimulus), sort(stimulus, decreasing = TRUE)),
        y = c(CI_02.5[order(stimulus)],
              CI_97.5[order(stimulus, decreasing = TRUE)]),
        col = adjustcolor('darkblue', alpha.f = 0.2), border = NA)
  lines(sort(stimulus),
        mod_mean[order(stimulus)],
        col = 'darkblue', lwd = 3)
})

with(subset(mod_pred, type %in% 'beta'), {
  polygon(x = c(sort(stimulus), sort(stimulus, decreasing = TRUE)),
        y = c(CI_02.5[order(stimulus)],
              CI_97.5[order(stimulus, decreasing = TRUE)]),
        col = adjustcolor('orange2', alpha.f = 0.2), border = NA)
  lines(sort(stimulus),
        mod_mean[order(stimulus)],
        col = 'orange2', lwd = 3)
})

```